

Multimodal Image Captioning Web Application

AI Project Case Study

Zachary Ziernicki

github.com/zziernicki-ui/Al-Image-Caption-App

Python · PyTorch · Hugging Face Transformers (BLIP) · Gradio

Executive Summary

This case study documents the design and implementation of a multimodal AI web application that generates natural-language captions for images. The application is built around **BLIP** (Bootstrapping Language-Image Pre-training), a pretrained vision-language transformer model from Hugging Face, and is deployed through a browser-based interface built with **Gradio**.

The project moves through four stages: setting up an isolated Python environment, building a command-line proof of concept to validate the model pipeline, refactoring that logic into a deployable function, and connecting it to a web interface. The result is a working tool that takes an uploaded image and returns a descriptive caption, along with a clear, demonstrable understanding of how each stage of the pipeline - preprocessing, inference, and decoding - fits together.

Project Objective

Problem Statement

Generating descriptive text for images is a recurring task across digital publishing, e-commerce, and content management; used for accessibility, search engine optimization, and organizing large image libraries. Done manually, this work is slow and difficult to scale.

Objective

Build a system that takes an image as input and automatically produces a natural-language caption using a pretrained vision-language transformer model, and make that capability accessible through a simple web interface.

Technical Architecture

The application is built around Hugging Face's BLIP model, which pairs a vision encoder with a language decoder so that visual information can be translated directly into text. The diagram below shows how an uploaded image moves through the system to produce a displayed caption.

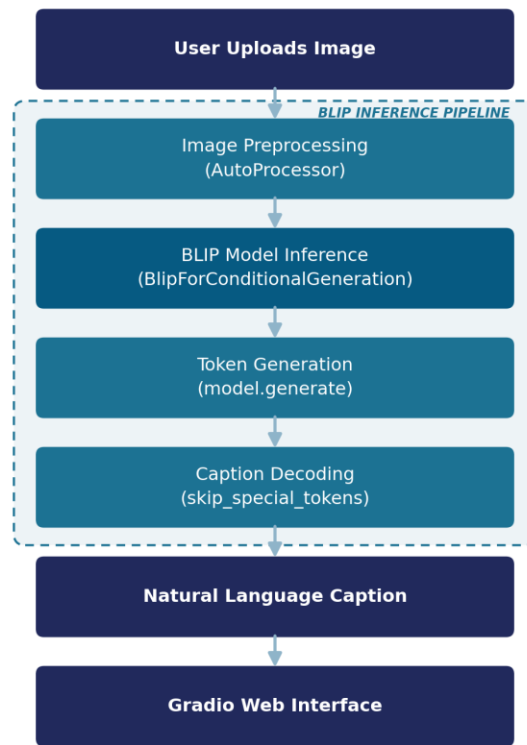


Figure 1. Image captioning pipeline, from upload to displayed caption.

Each stage of the pipeline plays a specific role:

- **AutoProcessor** — prepares the raw image for the model by converting it into the PyTorch tensor format BLIP expects.
- **BlipForConditionalGeneration** — the core model: a BLIP architecture configured for conditional text generation, taking image tensors as input and producing a sequence of output tokens.
- `model.generate()` — runs inference and generates the token sequence representing the caption. Inputs are passed using Python's dictionary-unpacking syntax (`**inputs`).
- `processor.decode(..., skip_special_tokens=True)` — converts the generated tokens back into readable text, filtering out model-specific control tokens.
- **Gradio** — wraps the captioning function in a browser-based interface, accepting an uploaded image and displaying the generated caption.

Development Workflow

Phase 1 — Environment Setup

Created an isolated Python environment using `virtualenv` to keep project dependencies separate from the system Python installation, then installed the core libraries:

- `transformers==4.38.2`
- `torch==2.2.1`

- `gradio==5.23.2`
- `langchain==0.1.11`
- `beautifulsoup4 (bs4)==0.0.2`
- `requests==2.31.0`

Phase 2 — Proof of Concept (`image_cap.py`)

Built a standalone script to validate the model pipeline before building any interface around it:

- Imported `AutoProcessor` and `BlipForConditionalGeneration` from the `transformers` library and loaded the pretrained BLIP checkpoint.
- Loaded a local image file and passed it through the processor with `return_tensors="pt"` to produce PyTorch tensors.
- Passed the resulting tensors into `model.generate(**inputs)`, unpacking the preprocessed inputs dictionary as keyword arguments.
- Decoded the generated token sequence into text using `processor.decode(output[0], skip_special_tokens=True)`.
- Verified the pipeline end-to-end: running `image_cap.py` against a test image returned a coherent caption describing the image's contents.

```
(my_env) theia@theia-zziernicki03: /home/project$ python3 image_cap.py
the image of a certificate for a professional
```

Figure 2. Output of `image_cap.py` run from the terminal.

Phase 3 — Web Interface (`image_captioning_app.py`)

Refactored the proof-of-concept logic into a reusable function and connected it to a Gradio interface:

- Installed Gradio and Pillow to support image handling in the interface layer.
- Wrapped the captioning logic in a single function that accepts an image — passed in by Gradio's image component as a NumPy array — and returns a caption string.
- Connected the function to a Gradio Interface, producing a browser-based tool where users upload an image and receive a generated caption with no command-line interaction required.

Phase 4 — Testing

Validated the deployed interface end-to-end using sample images, confirming that uploads are correctly preprocessed, inference runs without error, and a readable caption is displayed back to the user in the browser.

Results

The screenshots below show the deployed Gradio application running in the browser, tested against two different images: the certificate used during proof-of-concept testing, and an unrelated personal photo to confirm the model generalizes.

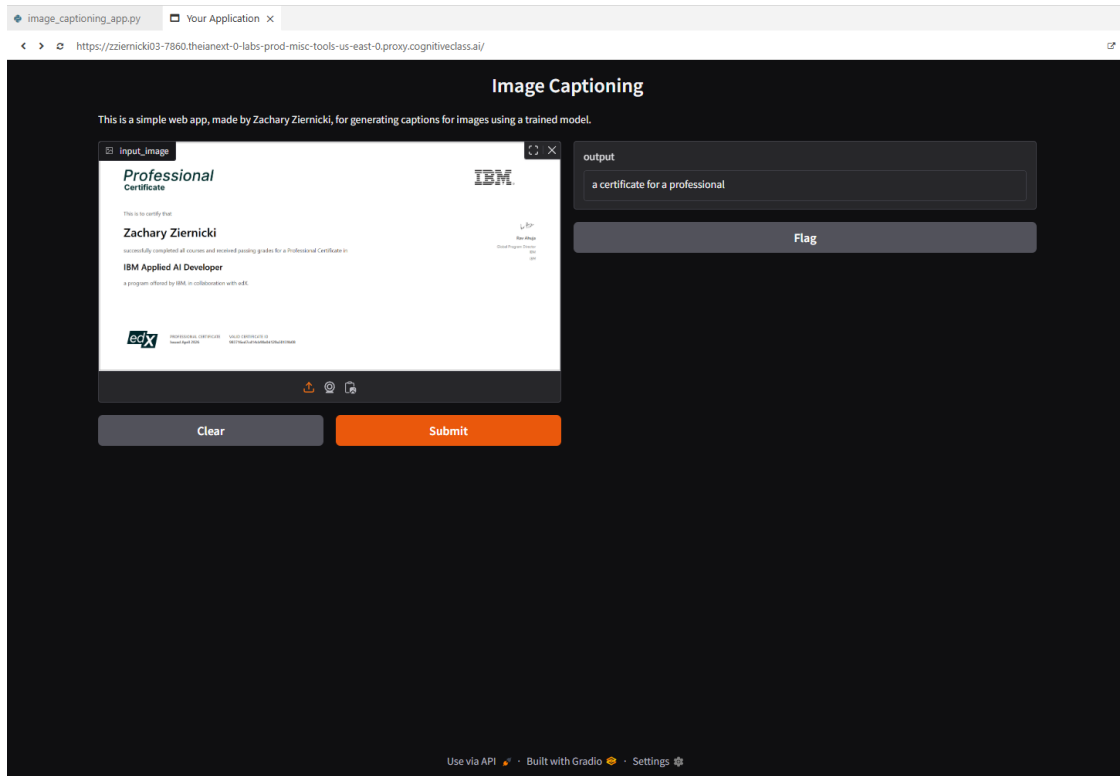


Figure 3. Application output for an uploaded image of a professional certificate.

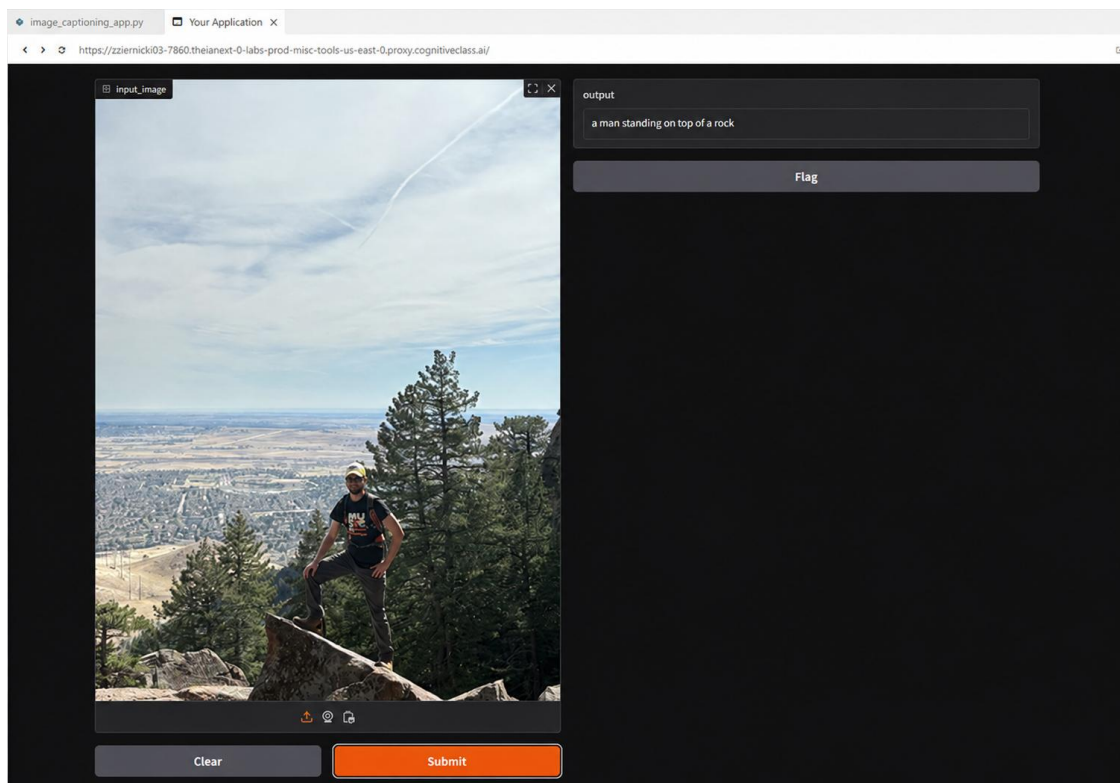


Figure 4. Application output for a personal photo, showing the model generalizes beyond the certificate test case.

The full source code is available at github.com/zziernicki-ui/Al-Image-Caption-App.

Core Technologies Used

- **Python** — primary application language.
- **PyTorch** — tensor operations and the execution backend for model inference.
- **Hugging Face Transformers** — provides the `AutoProcessor` and `BlipForConditionalGeneration` classes used to load and run the model.
- **BLIP** (Bootstrapping Language-Image Pre-training) — the pretrained vision-language model that performs the captioning.
- **Gradio** — framework used to build the browser-based interface.
- **pip / virtualenv** — dependency installation and environment isolation.

Engineering Concepts Demonstrated

- **Multimodal model architecture** — working with a model that combines a vision encoder and a language decoder in a single pipeline.
- **Tensor-based data representation** — converting raw image data into the tensor format a PyTorch model expects.
- **NLP tokenization and decoding** — generating token sequences and converting them back into readable text, including correct handling of special tokens.
- **Python patterns for ML APIs** — using dictionary unpacking (`**inputs`) to pass preprocessed data into model methods.
- **Iterative development** — progressing from a minimal proof-of-concept script to a packaged function to a deployed interface.
- **Environment management** — isolating dependencies with virtualenv.

Business Applications

The same underlying pipeline supports several practical use cases:

- **Accessibility** — automatically generating alt text for images on websites and in documents.
- **SEO and digital publishing** — producing descriptive captions and metadata for large image libraries.
- **Journalism and media** — reducing the time spent manually captioning images for online articles.
- **Digital asset management** — auto-tagging and organizing images within content repositories.

Future Enhancements

Natural next steps for extending this project include:

- URL-based image ingestion, so a user can caption an image by providing a link rather than uploading a file.
- Returning multiple candidate captions (e.g., the top three) so a user can choose the best fit.

- Comparing BLIP's output against other vision-language models to evaluate trade-offs in accuracy and speed.
- Wrapping the captioning function in a lightweight API (FastAPI or Flask) so it can be called from other applications.
- Adding prompt conditioning to generate captions tuned for specific use cases, such as SEO-oriented versus accessibility-oriented descriptions.

Key Takeaways

This project provided hands-on experience with the practical steps involved in taking a pretrained transformer model from a research artifact to a working application; environment setup, tensor-level data handling, model inference, output decoding, and UI integration.

This exercise also illustrates how a single pretrained multimodal model can be adapted to different contexts depending on the technical needs and use case. It was exciting getting to formulate ideas, where various tweaks allowed this structure to be integrated to meet different challenges, from accessibility tooling to content workflows.